
Attività svolta: Grafici e DataBase nell'interfaccia C# - Mini Supervisore

Aggiunte:

Ho aggiunto dei led per vedere lo stato del motore e dell'allarme sovrage. Poi ho aggiunto una progress bar per vedere meglio il numero di giriAttuali...

Grafici:

Ho inserito un grafico che si aggiorna da solo e si muove nel tempo (ogni secondo la linea muovendosi in avanti, mostrando sempre l'ultimo minuto di lavoro).

Sul grafico si vedono contemporaneamente due linee colorate per fare il confronto:

- **Linea Arancione (tratteggiata):** Indica la velocità che l'operatore ha chiesto al motore (il Setpoint).
- **Linea Blu (continua):** Indica i giri che il motore sta facendo veramente in quel momento.

DataBase (SQL):

Per implementare un database andare su Progetto → Gestione pacchetti NuGet e cercare **Microsoft.Data.Sqlite** e scaricarlo

Il database è un file che viene creato automaticamente alla prima connessione e SQLite è flessibile sui tipi.

Bisogna creare la classe **Repository.cs** e includere la libreria scaricata e per farlo basta scrivere **"using Microsoft.Data.Sqlite;"**

```

1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6  using Microsoft.Data.Sqlite;
7
8  namespace Mini_SupervisoreC_
9  {
10     0 riferimenti
11     public class Repository
12     {
13         private readonly string _connString = "Data Source=produzione.db";
14
15         0 riferimenti
16         public void inizializzaDb()
17         {
18             var conn = new SqliteConnection(_connString);
19             conn.Open();
20             var cmd = conn.CreateCommand();
21             cmd.CommandText = @"
22                 CREATE TABLE IF NOT EXISTS Produzione (
23                     Id            INTEGER PRIMARY KEY AUTOINCREMENT,
24                     Timestamp TEXT    NOT NULL,
25                     Giri          INTEGER NOT NULL,
26                     Motore        INTEGER NOT NULL,
27                     Ore           REAL NOT NULL
28                 );";
29             cmd.ExecuteNonQuery();
30         }
31     }
32 }

```

Inserire i dati (con parametri):

Non bisogna mai creare query concatenando delle stringhe. Concatenare dei valori in una query SQL espone a errori e a vulnerabilità, per questo si usano dei parametri, sia per sicurezza sia per correttezza dei tipi, che separano il comando dai dati in modo più sicuro e in modo da gestire automaticamente la formattazione dei tipi.

```

31     0 riferimenti
32     public void SalvaProduzione(LineaStato s)
33     {
34         var conn = new SqliteConnection(_connString);
35         conn.Open();
36         var cmd = conn.CreateCommand();
37         cmd.CommandText = @"
38             INSERT INTO Produzione (Timestamp, Giri, Motore, Ore)
39             VALUES ($ts, $giri, $motore, $ore);";
40         cmd.Parameters.AddWithValue("$ts", DateTime.Now.ToString("o"));
41         cmd.Parameters.AddWithValue("$giri", s.GiriAttuali);
42         cmd.Parameters.AddWithValue("$motore", s.MotoreAttivo ? 1 : 0);
43         cmd.Parameters.AddWithValue("$ore", s.OreFunzionamento);
44         cmd.ExecuteNonQuery();
45     }

```

Tabella allarmi e organizzazione codice:

Aggiungiamo una seconda tabella per gli allarmi nella **InizializzaDb()**:

```
0 riferimenti
15 public void InizializzaDb()
16 {
17     var conn = new SqlConnection(_connString);
18     conn.Open();
19     var cmd = conn.CreateCommand();
20     cmd.CommandText = @"
21         CREATE TABLE IF NOT EXISTS Produzione (
22             Id          INTEGER PRIMARY KEY AUTOINCREMENT,
23             Timestamp TEXT    NOT NULL,
24             Giri        INTEGER NOT NULL,
25             Motore      INTEGER NOT NULL,
26             Ore         REAL NOT NULL
27         );
28
29         CREATE TABLE IF NOT EXISTS Allarmi (
30             Id          INTEGER PRIMARY KEY AUTOINCREMENT,
31             Timestamp TEXT    NOT NULL,
32             Tipo        TEXT NOT NULL,
33             Descrizione TEXT
34         );";
35
36     cmd.ExecuteNonQuery();
37 }
38
```

Ora scriviamo una funzione uguale a **SalvaProduzione()** ma per salvare l'allarme:

```
0 riferimenti
54 public void SalvaAllarme(string tipo, string descr)
55 {
56     var conn = new SqlConnection(_connString);
57     conn.Open();
58     var cmd = conn.CreateCommand();
59     cmd.CommandText = @"
60         INSERT INTO Allarmi (Timestamp, Tipo, Descrizione)
61         VALUES ($ts, $tipo, $descr);";
62     cmd.Parameters.AddWithValue("$ts", DateTime.Now.ToString("o"));
63     cmd.Parameters.AddWithValue("$tipo", tipo);
64     cmd.Parameters.AddWithValue("$descr", descr);
65     cmd.ExecuteNonQuery();
66 }
67
```